

Pilot Study Empirical Validation: Optimal vs. Sub-Optimal Code

Combination Sum II

EffiBench Optimal:

Accepted 176 / 176 testcases passed

Sarah Wilson submitted at Jul 03, 2026 18:58

Analysis

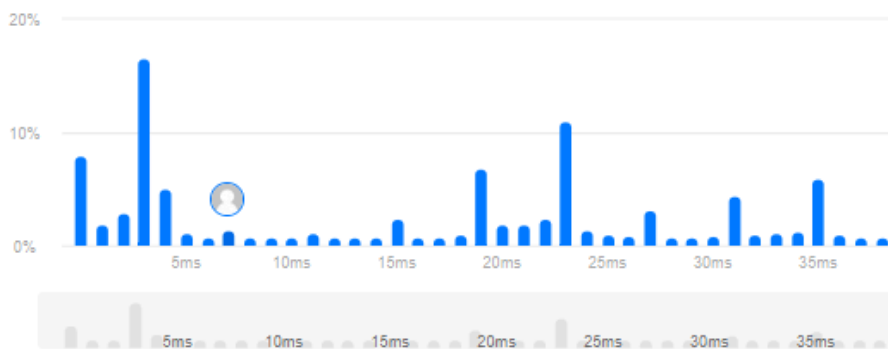
Solution

Runtime

7 ms | Beats 63.85%

Memory

19.62 MB | Beats 12.20%



Code | Python3

```
1
2
3 class Solution:
4     def combinationSum2(self, candidates: List[int], target: int) -> List[List[int]]:
5         def dfs(i: int, s: int):
6             if s == 0:
7                 ans.append(t[:])
8                 return
9             if i >= len(candidates) or s < candidates[i]:
10                return
11            for j in range(i, len(candidates)):
12                if j > i and candidates[j] == candidates[j - 1]:
13                    continue
14                t.append(candidates[j])
15                dfs(j + 1, s - candidates[j])
16                t.pop()
17
18            candidates.sort()
19            ans = []
20            t = []
21            dfs(0, target)
22            return ans
```

View less

Exceeds LeetCode time limits but is functionally valid as a sub-optimal solution (verified by humans).

 Editorial Use Testcase[illegible]

30

```

1 class Solution:
2     def combinationSum2(self, candidates: List[int], target: int) -> List[List[int]]:
3         # Sub-optimal: Exponential brute-force recursion without pruning
4         ans = []
5         def dfs(idx, current_path, current_sum):
6             if current_sum == target:
7                 if sorted(current_path) not in ans:
8                     ans.append(sorted(current_path))
9             return
10            if current_sum > target or idx >= len(candidates):
11                return
12            for i in range(idx, len(candidates)):
13                dfs(i + 1, current_path + [candidates[i]], current_sum + candidates[i])
14        dfs(0, [], 0)
15        return ans

```

⌵ [View less](#)

Remove Duplicates From Sorted Array II

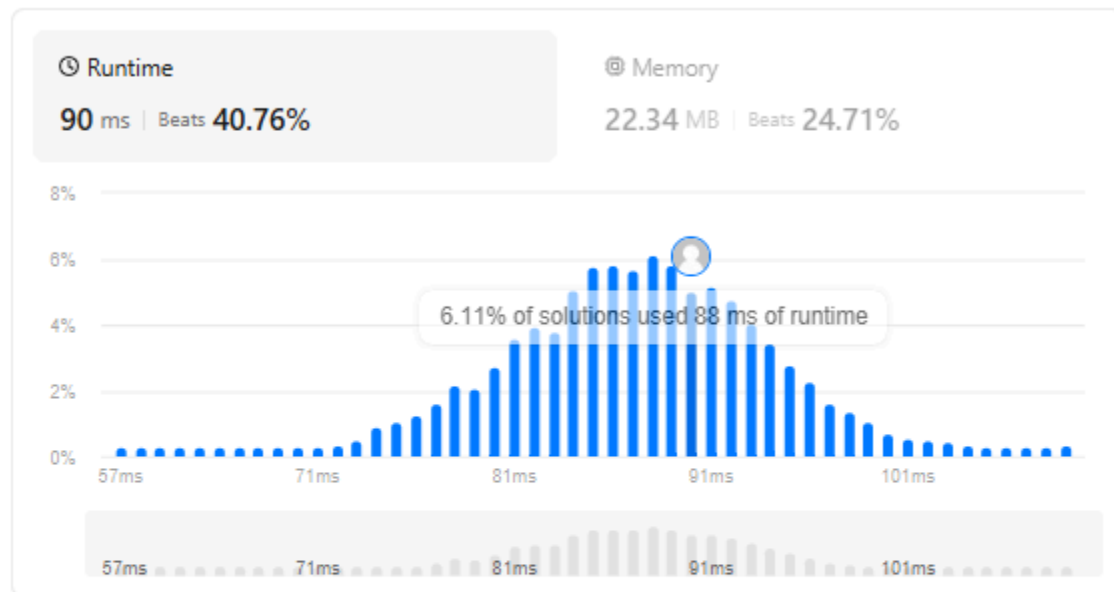
EffiBench Optimal:

Accepted 170 / 170 testcases passed

 Sarah Wilson submitted at Jul 03, 2026 18:59

 Analysis

 Solution



Code | Python3

```
1 class Solution:
2     def removeDuplicates(self, nums: List[int]) -> int:
3         k = 0
4         for x in nums:
5             if k < 2 or x != nums[k - 2]:
6                 nums[k] = x
7                 k += 1
8         return k
```

Gemini-Generated Sub-optimal:

Accepted 170 / 170 testcases passed

 Sarah Wilson submitted at Jul 03, 2026 18:59

 Analysis

 Solution

⌚ Runtime

5149 ms | Beats 5.12%

💾 Memory

22.05 MB | Beats 64.69% 🌿



Code | Python3

```
1 class Solution:
2     def removeDuplicates(self, nums: List[int]) -> int:
3         # Sub-optimal: O(n^2) approach using repeated list popping
4         i = 0
5         while i < len(nums):
6             if nums.count(nums[i]) > 2:
7                 nums.pop(i)
8             else:
9                 i += 1
10        return len(nums)
```

 View less

Finding 3-Digit Even Numbers

EffiBench Optimal:

Needed to add `from collections import Counter` in LeetCode.

Accepted 79 / 79 testcases passed

Sarah Wilson submitted at Jul 03, 2026 19:11

Analysis

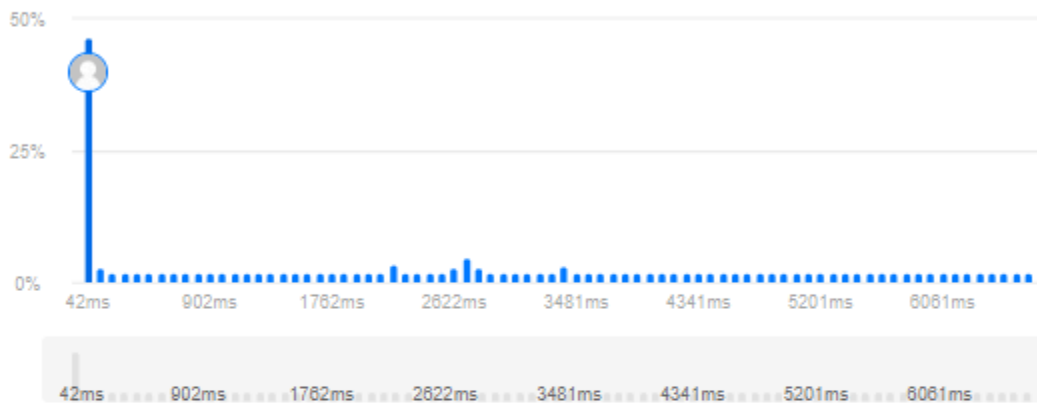
Solution

Runtime

79 ms | Beats 55.91%

Memory

19.49 MB | Beats 28.18%



Code | Python3

```
1 from collections import Counter
2
3 class Solution:
4     def findEvenNumbers(self, digits: List[int]) -> List[int]:
5         ans = []
6         counter = Counter(digits)
7         for i in range(100, 1000, 2):
8             t = []
9             k = i
10            while k:
11                t.append(k % 10)
12                k //= 10
13            cnt = Counter(t)
14            if all([counter[i] >= cnt[i] for i in range(10)]):
15                ans.append(i)
16        return ans
```

View less

Gemini-Generated Sub-optimal:

Accepted 79 / 79 testcases passed

 Sarah Wilson submitted at Jul 03, 2026 19:02

 Analysis

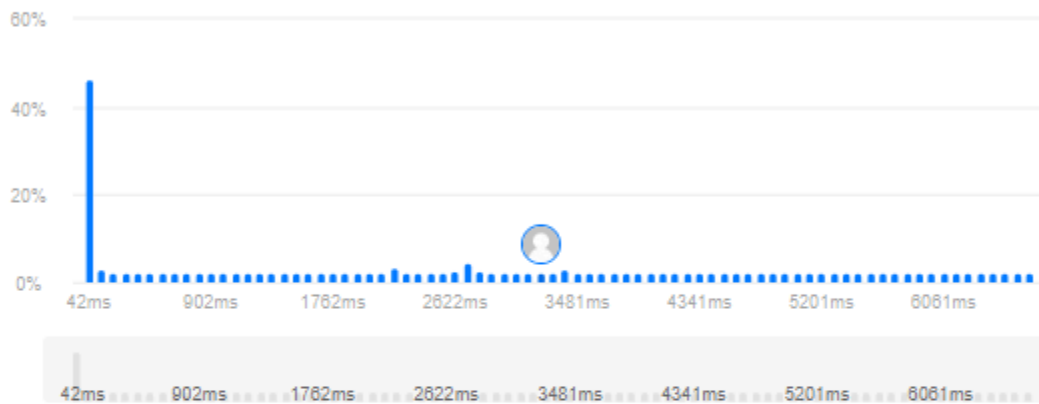
 Solution

⌚ Runtime

3276 ms | Beats 23.21%

@ Memory

19.25 MB | Beats 79.27% 🏆



Code | Python3

```
1 class Solution:
2     def findEvenNumbers(self, digits: List[int]) -> List[int]:
3         # Sub-optimal: O(n^3) nested loop checking all digit combinations pe
4         ans = set()
5         n = len(digits)
6         for i in range(n):
7             for j in range(n):
8                 for k in range(n):
9                     if i != j and j != k and i != k:
10                        val = digits[i] * 100 + digits[j] * 10 + digits[k]
11                        if val >= 100 and val % 2 == 0:
12                            ans.add(val)
13         return sorted(list(ans))
```

⤴ View less ⤵

Is Same Tree

LeetCode's execution runtime measurement can be a bit blunt for very lightweight tasks. Both solutions return in 0ms because the test cases are small (maximum of 100 nodes per tree). Modern CPUs process 100 operations in fractions of a microsecond, rounding down to 0ms on the platform. The serialization method is sub-optimal based on memory/space complexity, but more importantly, from a pure algorithmic perspective, the serialization approach is drastically sub-optimal compared to structural recursion, and this is verifiable by humans.

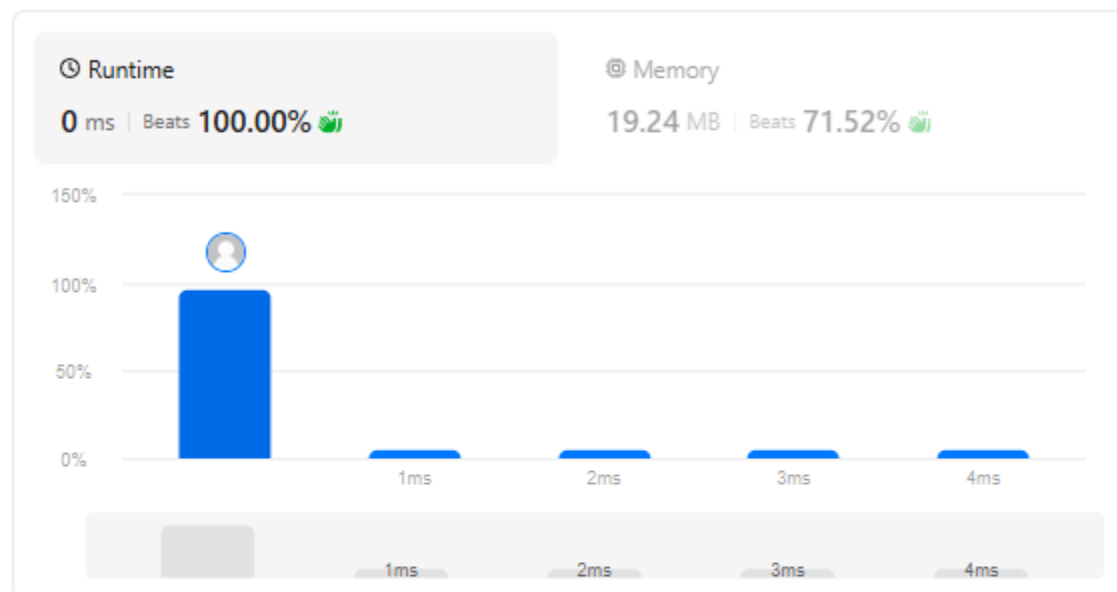
EffiBench Optimal:

Accepted 67 / 67 testcases passed

 Sarah Wilson submitted at Jul 03, 2026 19:00

 Analysis

 Solution



Code | Python3

```
1 class Solution:
2     def isSameTree(self, p: Optional[TreeNode], q: Optional[TreeNode]) -> bool
3         if p == q:
4             return True
5         if p is None or q is None or p.val != q.val:
6             return False
7         return self.isSameTree(p.left, q.left) and self.isSameTree(p.right, q.
```

Gemini-Generated Sub-optimal:

Accepted 67 / 67 testcases passed

Sarah Wilson submitted at Jul 03, 2026 19:00

Analysis

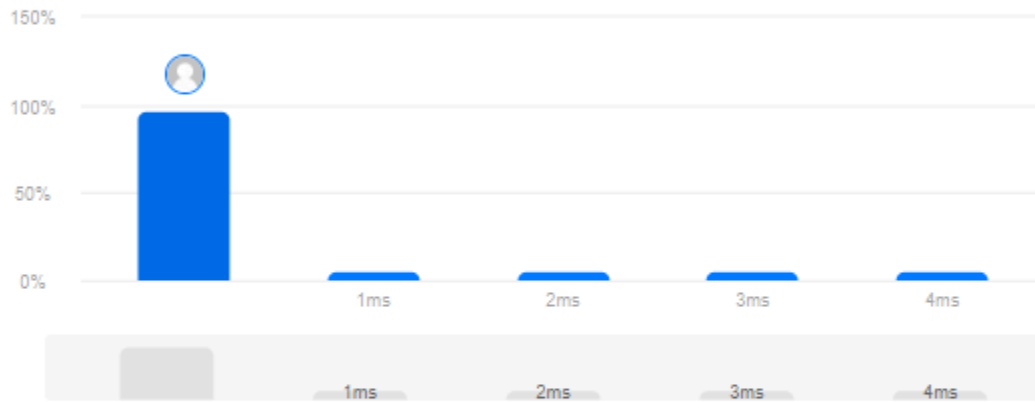
Solution

Runtime

0 ms | Beats 100.00%

Memory

19.35 MB | Beats 30.69%



Code | Python3

```
1 class Solution:
2     def isSameTree(self, p: Optional[TreeNode], q: Optional[TreeNode]) -> bool
3         # Sub-optimal: String serialization comparison instead of structural
4         def serialize(root):
5             if not root: return "None"
6             return f"{root.val},{serialize(root.left)},{serialize(root.right)}"
7         return serialize(p) == serialize(q)
```

Minimum Operations to Reduce an Integer to 0

EffiBench Optimal:

Accepted 41 / 41 testcases passed

Sarah Wilson submitted at Jul 03, 2026 19:04

Analysis

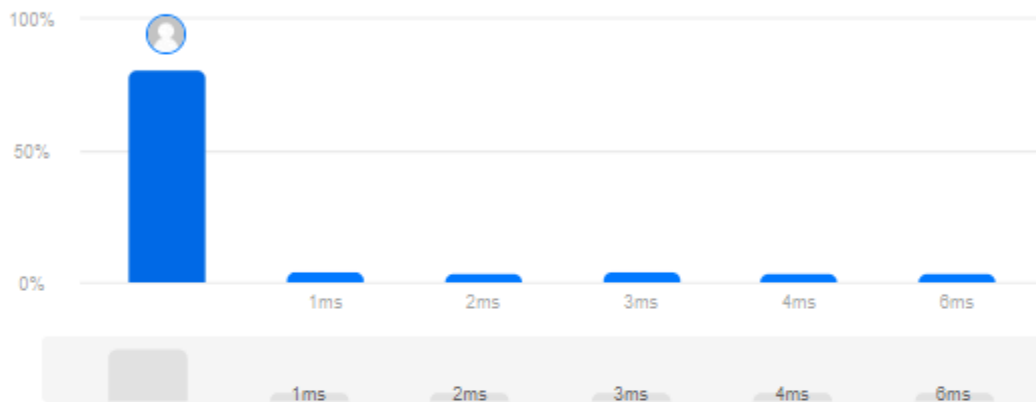
Solution

Runtime

0 ms | Beats 100.00%

Memory

19.28 MB | Beats 66.46%



Code | Python3

```
1 class Solution:
2     def minOperations(self, n: int) -> int:
3         ans = cnt = 0
4         while n:
5             if n & 1:
6                 cnt += 1
7             elif cnt:
8                 ans += 1
9                 cnt = 0 if cnt == 1 else 1
10            n >>= 1
11        if cnt == 1:
12            ans += 1
13        elif cnt > 1:
14            ans += 2
15        return ans
```

View less

Gemini-Generated Sub-optimal:

The sub-optimal solution is a mathematically valid greedy algorithm for a restricted subset of the rules where you are only allowed to subtract powers of 2 (effectively counting and clearing individual binary "1" bits). It fails the broader LeetCode 2571 problem because it lacks the look-ahead capability to add a power of 2, which would collapse multiple consecutive bits into a single higher bit using a binary carry.

Therefore, it does not pass basic tests on LeetCode, but it is still a valid sub-optimal code snippet for the purposes of our study. This aligns with our evaluation criteria, where we wanted the sub-optimal snippets to be consistently generated across all questions pass@1 with minimal human intervention, capturing raw LLM behavior without manual optimization.